AI Software Engineering - Blocco 2

Lezione 6

Clean / Hexagonal Architecture: creare confini e test seams stabili

Domanda centrale: abbiamo imparato a specificare e testare il comportamento. Come progettiamo ora il software affinché quel comportamento possa essere verificato senza dipendere direttamente da framework, database, HTTP o servizi esterni?

Spina dorsale del corso: Domain -> Requirements -> Spec -> Test -> Agent -> Verify

Questa dispensa è autosufficiente: riprende solo il minimo necessario dalle lezioni precedenti e approfondisce il tema architetturale della Lezione 6.

1. Perché questa lezione esiste

Nelle lezioni precedenti abbiamo costruito una catena di affidabilità: domain evidence, business rule, requirement, acceptance example, test e implementazione. Questa catena però può rompersi in un punto molto concreto: il codice può diventare difficile da verificare se il comportamento importante è mescolato con dettagli tecnici.

Il problema tipico è questo: una regola di dominio viene implementata dentro un controller, dentro un ORM model, dentro una route handler o dentro un job che chiama direttamente un servizio esterno. Il test diventa allora più vicino a un test del cablaggio tecnico che a una prova della regola. Può essere verde, ma fragile; può essere realistico, ma lento; può coprire il framework, ma non isolare la policy.

Clean Architecture e Hexagonal Architecture rispondono allo stesso problema ingegneristico: separare ciò che deve restare stabile - le policy di dominio e i casi d'uso - da ciò che può cambiare - database, UI, HTTP, code framework, servizi esterni, storage, cloud provider.

Principio della lezione

> Un buon confine architetturale rende il comportamento verificabile da una fonte indipendente dall'implementazione e permette a un agente AI di modificare una parte del sistema senza dover reinterpretare tutto il dominio.

2. Clean Architecture in una frase

Clean Architecture mette al centro il dominio e i casi d'uso. I dettagli tecnici stanno all'esterno. La regola fondamentale è la Dependency Rule: le dipendenze del codice sorgente puntano verso l'interno. In altre parole, il codice interno non deve conoscere nomi, tipi o dettagli del codice esterno.

Questo non significa che il runtime non possa chiamare un database o un servizio remoto. Significa che il codice che esprime la policy non deve dipendere da quei dettagli. Il caso d'uso può parlare con una interfaccia, una porta, un contratto. L'adapter concreto, che sta fuori, implementa quel contratto usando SQL, HTTP, filesystem o un framework specifico.

Zona	Responsabilità	Cosa non deve conoscere
Domain model	Regole, invarianti, entità, value object, decisioni di dominio	HTTP, SQL, ORM, framework, controller
Use case / application service	Orchestrazione applicativa: input, regole, porte, output	Implementazioni concrete di DB, API esterne, UI
Ports	Interfacce orientate allo scopo: repository, notifier, payment gateway	Tecnologia concreta scelta
Adapters	Traduzione da/verso tecnologia concreta: controller, repository SQL, client API	Business policy non necessaria

3. Hexagonal Architecture in una frase

Hexagonal Architecture, o Ports & Adapters, descrive lo stesso principio con una metafora diversa. Non ci sono "sei livelli". L'esagono serve a ricordare che l'applicazione ha un dentro e un fuori: dentro stanno dominio e casi d'uso; fuori stanno gli attori tecnici che guidano o servono l'applicazione.

Una porta è una conversazione intenzionale con l'applicazione. Un adapter traduce una tecnologia concreta nel linguaggio della porta. Lo stesso caso d'uso può essere invocato da HTTP, CLI, batch, test harness o messaggi di coda. Allo stesso modo può salvare dati tramite

SQL, memoria, file, API remota o mock adapter, purché il contratto della porta resti stabile.

La domanda non è: “quante cartelle creo?”. La domanda è: “quali conversazioni voglio rendere esplicite e verificabili?”.

4. Dependency Inversion: non confonderla con Dependency Injection

Dependency Inversion è un principio di design: i moduli ad alto livello non devono dipendere dai dettagli a basso livello; entrambi dovrebbero dipendere da astrazioni. Dependency Injection è una tecnica di wiring: un oggetto riceve le dipendenze dall’esterno invece di costruirle internamente. Le due cose collaborano, ma non sono sinonimi.

Concetto	Domanda a cui risponde	Esempio
DIP	Da cosa deve dipendere il codice importante?	Il caso d’uso dipende da TicketRepository, non da MySqlTicketRepository
DI	Come fornisco la dipendenza concreta?	Nel composition root collego TicketRepository a MySqlTicketRepository
IoC	Chi controlla il flusso?	Il framework chiama il controller quando arriva una richiesta

5. Use case / application service: il seam principale

Il test seam più importante è spesso il caso d’uso. Un use case riceve un input applicativo, carica ciò che serve tramite porte, invoca il dominio, persiste il risultato tramite porte e restituisce un outcome. Non dovrebbe contenere dettagli di controller, request HTTP, response JSON, ORM query o API SDK.

Esempio concettuale: chiudere un ticket.

Regola: un ticket può essere chiuso solo se non è già chiuso e se viene fornita una motivazione valida. Alla chiusura si registra un evento e, se richiesto, si invia una notifica.

Possibile decomposizione:

• Domain model: Ticket, TicketStatus, ClosingReason, invariant “non chiudere due volte”.

• Use case: CloseTicketUseCase.

• Ports: TicketRepository, NotificationPort, EventRecorder.

• Adapters: SqlTicketRepository, EmailNotificationAdapter, InMemoryTicketRepository per test.

• Controller: traduce HTTP request in CloseTicketCommand e traduce risultato in HTTP response.

6. Esempio in TypeScript

```
interface TicketRepository {
  get(id: TicketId): Promise
  save(ticket: Ticket): Promise
}

interface NotificationPort {
  notifyTicketClosed(ticket: Ticket): Promise
}

class CloseTicketUseCase {
  constructor(
```

```
    private tickets: TicketRepository,
    private notifications: NotificationPort
  ) {}

  async execute(cmd: CloseTicketCommand): Promise {
    const ticket = await this.tickets.get(cmd.ticketId)
    ticket.close(cmd.reason)          // regola nel dominio
    await this.tickets.save(ticket)   // porta, non SQL diretto
    if (cmd.notify) await this.notifications.notifyTicketClosed(ticket)
    return { status: 'CLOSED' }
  }
}
```

Test senza framework, HTTP o database reale:

```
class InMemoryTicketRepository implements TicketRepository {
  constructor(private ticket: Ticket) {}
  async get(id: TicketId) { return this.ticket }
  async save(ticket: Ticket) { this.ticket = ticket }
}

class SpyNotificationPort implements NotificationPort {
  public sent = 0
  async notifyTicketClosed(ticket: Ticket) { this.sent++ }
}

it('closes a planned ticket and notifies when requested', async () => {
  const repo = new InMemoryTicketRepository(plannedTicket())
  const notifier = new SpyNotificationPort()
  const useCase = new CloseTicketUseCase(repo, notifier)

  const result = await useCase.execute({ ticketId: 'T-1', reason: 'fixed', notify: true })

  expect(result.status).toBe('CLOSED')
  expect((await repo.get('T-1')).isClosed()).toBe(true)
  expect(notifier.sent).toBe(1)
})
```

7. Esempio equivalente in PHP

```
interface TicketRepository {
    public function get(TicketId $id): Ticket;
    public function save(Ticket $ticket): void;
}

final class CloseTicketUseCase {
    public function __construct(
        private TicketRepository $tickets,
        private NotificationPort $notifications,
    ) {}

    public function execute(CloseTicketCommand $command): CloseTicketResult {
        $ticket = $this->tickets->get($command->ticketId);
        $ticket->close($command->reason);
        $this->tickets->save($ticket);

        if ($command->notify) {
            $this->notifications->ticketClosed($ticket);
        }

        return new CloseTicketResult('CLOSED');
    }
}
```

In Laravel, Symfony, CodeIgniter o qualunque altro framework, il controller dovrebbe restare un adapter. Il controller non deve diventare il posto in cui vive la policy di chiusura del ticket.

8. Esempio equivalente in Python

```
from typing import Protocol

class TicketRepository(Protocol):
    def get(self, ticket_id: str) -> Ticket: ...
    def save(self, ticket: Ticket) -> None: ...

class CloseTicketUseCase:
    def __init__(self, tickets: TicketRepository, notifications: NotificationPort):
        self.tickets = tickets
```

```
        self.notifications = notifications

    def execute(self, command: CloseTicketCommand) -> CloseTicketResult:
        ticket = self.tickets.get(command.ticket_id)
        ticket.close(command.reason)
        self.tickets.save(ticket)
        if command.notify:
            self.notifications.ticket_closed(ticket)
        return CloseTicketResult(status='CLOSED')
```

In Django o FastAPI, la view/route handler può istanziare o ricevere il caso d'uso, ma non dovrebbe incorporare la policy di dominio. In questo modo i test del caso d'uso restano indipendenti dal web framework.

9. Rapporto con TDD, BDD, ATDD e Specification by Example

Clean/Hexagonal Architecture rende praticabile il modello a due loop visto nella lezione precedente:

• Outer loop: acceptance example o acceptance test stabilisce il comportamento atteso dal punto di vista del dominio.

• Inner loop: TDD guida il design interno del dominio e del caso d'uso.

• Adapter tests: verificano che gli adapter reali rispettino i contratti delle porte.

• E2E tests: verificano pochi percorsi critici di cablaggio, non ogni regola.

Questo evita un errore frequente: usare solo test end-to-end per provare regole di dominio. I test E2E sono utili, ma costosi e fragili; non dovrebbero essere l'unico modo per sapere se una business rule funziona.

10. Rapporto con gli agenti AI

Per un coding agent, i confini architetturali sono confini cognitivi e operativi. Un agente lavora meglio quando riceve un compito con scope ridotto, contratti chiari, file rilevanti e test espliciti. Clean/Hexagonal Architecture aiuta perché separa use case, porte, adapter e dominio.

Prompt migliore per un agente:

> "Implementa CloseTicketUseCase rispettando la porta TicketRepository già definita. Non modificare controller, migration o adapter reali. Aggiungi test unitari del caso d'uso con InMemoryTicketRepository. Non cambiare la policy di dominio senza aggiornare SPEC.md e ADR."

Prompt peggiore:

> "Sistema la chiusura ticket in tutta l'app."

Il primo prompt preserva authority e confini. Il secondo spinge l'agente a cercare, dedurre e forse inventare il dominio.

11. Quando usare questo approccio

• Quando il comportamento di dominio ha valore economico o rischio operativo.

• Quando il sistema deve evolvere e cambieranno DB, API, UI, code framework o integrazioni.

• Quando vuoi test rapidi e indipendenti dalla infrastruttura.

• Quando lavori con agenti AI e vuoi ridurre il rischio di modifiche casuali.

• Quando la decision density è alta: molte regole, eccezioni, invarianti, workflow.

Quando non forzarlo

• Script temporanei o automazioni usa-e-getta.

• CRUD banale senza regole significative.

• Prototipi esplorativi in cui il problema non è ancora compreso.

• Contesti in cui il team non sa mantenere le astrazioni e finisce per creare boilerplate vuoto.

• Casi in cui la separazione nasce da paura del cambiamento, non da esigenze reali.

12. Failure mode e anti-pattern

Anti-pattern	Sintomo	Correzione
Framework al centro	La business rule vive in controller, model ORM o request handler	Spostare la policy in domain/use case e lasciare il controller come adapter
Repository finto	Un'interfaccia replica l'ORM senza linguaggio di dominio	Definire porte per conversazioni intenzionali, non per nascondere meccanicamente il DB
Overengineering	Dieci livelli per un CRUD senza regole	Applicare il processo in proporzione a rischio e decision density
Microservizi travestiti	Si crede che Hexagonal implichi deployment distribuito	Separare architettura logica da deployment fisico
Test solo E2E	Ogni verifica richiede browser, DB e servizi reali	Spostare molte prove su use case test e contract test degli adapter
Porta instabile	La porta espone dettagli SQL/HTTP invece del linguaggio applicativo	Riformulare la porta intorno allo scopo: "save ticket", "notify closed", non "execute query"

13. Differenza tra architettura logica e deployment

Hexagonal Architecture non significa microservizi. Clean Architecture non significa creare molti repository o molti servizi deployati. Puoi avere una architettura pulita dentro un monolite modulare. Puoi anche avere microservizi disordinati senza confini puliti. La domanda architetturale è: dove sono i confini logici? La domanda di deployment è: in quanti processi o servizi deployo il sistema? Sono due decisioni diverse.

Regola pratica: prima disegna i confini logici; solo dopo decidi se qualcuno di quei confini merita un deployment separato.

14. Esercizio pratico

Obiettivo: trasformare una feature in un use case testabile senza dipendere da framework, DB reale o HTTP.

Scenario: "Chiudere un ticket di manutenzione solo se non è già chiuso e se contiene una motivazione valida. Alla chiusura va registrato un evento e, se richiesto, inviata una notifica."

Passi:

• Scrivi 3 esempi Given/When/Then: chiusura valida, ticket già chiuso, motivazione mancante.

• Disegna il caso d'uso CloseTicketUseCase.

• Definisci almeno due porte: TicketRepository e NotificationPort. Aggiungi EventRecorder se vuoi tracciare eventi.

• Implementa fake/in-memory adapter per i test.

• Scrivi un test del caso d’uso che non usi HTTP, database reale o framework.

• Scrivi un breve ADR: “Perché abbiamo messo la policy fuori dal controller?”

Criteri di verifica:

• Il dominio non importa framework.

• Il caso d’uso non conosce SQL, HTTP o SDK esterni.

• Il controller è solo un adapter.

• Il test dimostra la regola senza infrastruttura reale.

• L’agente AI può modificare il caso d’uso senza toccare adapter non correlati.

15. Takeaway finale

Clean/Hexagonal Architecture non serve a “fare architettura elegante”. Serve a proteggere il comportamento importante dai dettagli instabili, creando confini che rendono il software verificabile, modificabile e leggibile anche dagli agenti AI. In questo corso, la sua funzione è chiara: collegare Spec e Test a una struttura del codice che permetta davvero Verify.

Frase da ricordare:

> Il framework deve eseguire il tuo dominio, non possederlo.

Ponte verso la Lezione 7:

Una volta creati porte e adapter, nasce la domanda successiva: come dimostriamo che un adapter concreto rispetta davvero la porta? E come proviamo che una regola vale su molti casi, non solo sui tre esempi scelti? Qui entreranno Contract Testing e Property-Based Testing.

Fonti principali

• Alistair Cockburn, “Hexagonal Architecture / Ports & Adapters”, 2005, sito autore.

• Robert C. Martin, “The Clean Architecture”, Clean Coder Blog, 2012.

• Martin Fowler, “Inversion of Control Containers and the Dependency Injection pattern”, 2004.

• Martin Fowler, “DIP in the Wild”, 2013.

• Microsoft Learn, “Common web application architectures”, sezione Clean Architecture, consultata nel 2026.